ANNEXURE – 16

Nirma University
Institute of Technology

Computer Engineering Department

Course Policy

MCA Branch

Semester: II, Academic Year: 2025-26 Term: Even

Course Code & Name	:	6CS151CC25 Design and Analysis of Algorithms
Credit Details	:	3hrs lectures 2 hrs. labs 4 credits
Course Co-ordinator	:	Dr Lata Gohil
Contact No. & Email	:	9426359352 lata.gohil@nirmauni.ac.in
Office	:	9563
Visiting Hours	:	Free slots in Time table
Course Blog	:	-
Course Faculty	:	Dr Lata Gohil, Mr Mitrajsinh Chudasama

Introduction to Course:

Design and Analysis of Algorithms is the most important basic course in any computer science curriculum. It is vital for every computer science student to be fluent with algorithms and their analysis.

The objective of this course is to build a solid foundation of the most important fundamental subject in computer science. Creative thinking is essential to algorithm design. Algorithm analysis and verification demands sound mathematical acumen and programming skills.

Course Learning Outcomes:

1. identify the appropriate data structure to design an efficient algorithm for the given problem
2. implement various techniques for searching and sorting
3. apply appropriate algorithmic technique to solve a given problem
4. analyze performance of algorithms and estimate their worst-case and average-case behavior

Syllabus:

Syllabus:	**Teaching Hours: 45**
Unit I **Elementary algorithm:** introduction to notations for program, efficiency of algorithms.	**2**
Unit II **Asymptotic Notation:** a notation for "the order of", other asymptotic notation, conditional asymptotic notation. **Analysis of algorithms:** analyzing control structures: sequencing, "for" loops, recursive calls, "while" and "repeat" loops, using a barometer, amortized analysis.	**10**
Unit III **Greedy algorithms:** graphs: minimum spanning trees-kruskal's algorithm, prim's algorithm, graphs: shortest paths. Knapsack problem, **scheduling:** minimizing time in the system, scheduling with deadlines.	**8**
Unit IV **Divide-and-conquer:** multiplying large integers, binary search, sorting: sorting by merging, quick sort, finding the median, matrix multiplication, and exponentiation.	**8**
Unit V **Dynamic programming:** making change, the principle of optimality, the knapsack problem, shortest path, chained matrix multiplication, approaches using recursion, memory functions.	**8**
Unit VI **Backtracking & branch-and-bound:** basic idea, 8-queens problem, graph coloring, hamiltonian cycles, knapsack problem.	**7**
Unit VII **Introduction to np-completeness:** the class P and NP, polynomial reduction, np-completeness problem, np-hard problems.	**2**

Self-study:

The self study contents will be declared at the commencement of semester. Around 10% of the questions will be asked from self study contents.

<u>Laboratory details</u>: (List of Experiments, Schedule, assessment policy)

Assessment Policy: Assessment of Program(s) and Viva-voce

<u>List of Practical</u>

Sr. No.	Name of Practical	Hour(s)	CLOs
1.	**Topic:** Array and Matrix 1. <u>Rotate Array</u> Given an integer array *nums*, rotate the array to the right by *k* steps, where *k* is non-negative. https://leetcode.com/problems/rotate-array/ 2. <u>Median of Two Sorted Arrays</u> Given two sorted arrays *nums1* and *nums2* of size *m* and *n* respectively, return the median of the two sorted arrays. The overall run time complexity should be *O(log (m+n))*. https://leetcode.com/problems/median-of-two-sorted-arrays/ 3. <u>Find First and Last Position of Element in Sorted Array</u> Given an array of integers *nums* sorted in non-decreasing order, find the starting and ending position of a given *target* value. If *target* is not found in the array, return [-1, -1]. You must write an algorithm with *O(log n)* runtime complexity. https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/ 4. <u>Spiral Matrix</u> Given an *m x n* matrix, return all elements of the matrix in spiral order. https://leetcode.com/problems/spiral-matrix/ 5. <u>Minimum Path Sum</u> Given an *m x n* grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path. Note: You can only move either down or right at any point in time. https://leetcode.com/problems/minimum-path-sum/	2	**CLO - 2, 4**
2.	**Topic:** Sorting Algorithms 1. Implement iterative and full recursive versions of the following sorting techniques and repeat the experiment for larger values of n with different input patterns, and plot the comparison graph of the number of elements versus the execution time taken. The elements can be read from a file or can be generated using a random number generator. 1. Selection sort 2. Insertion sort 3. Bubble sort	4	**CLO - 2, 3, 4**

	2. Given an array of integers *nums*, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in *O(nlog(n))* time complexity and with the smallest space complexity possible.		
3.	**Topic:** Searching Algorithms 1. You are given an m x n binary matrix mat of 1's (representing soldiers) and 0's (representing civilians). The soldiers are positioned in front of the civilians. That is, all the 1's will appear to the left of all the 0's in each row. A row *i* is weaker than a row *j* if one of the following is true: • The number of soldiers in row *i* is less than the number of soldiers in row *j*. • Both rows have the same number of soldiers and *i* < *j*. Return the indices of the *k* weakest rows in the matrix ordered from weakest to strongest. https://leetcode.com/problems/the-k-weakest-rows-in-a-matrix/ 2. You are given a sorted array consisting of only integers, where every element appears exactly twice, except for one element, which appears exactly once. Return the single element that appears only once. Your solution must run in *O(log n)* time and *O(1)* space. https://leetcode.com/problems/single-element-in-a-sorted-array/ 3. Nearly everyone has used the Multiplication Table. The multiplication table of size *m* x *n* is an integer matrix *mat* where *mat*[i][j] == i * j (1-indexed). Given three integers *m*, *n*, and *k*, return the kth smallest element in the *m* x *n* multiplication table. https://leetcode.com/problems/kth-smallest-number-in-multiplication-table/	2	**CLO - 2, 3, 4**
4.	**Topic:** Linked List 1. Given the head of a linked list, reverse the nodes of the list *k* at a time, and return the modified list. *k* is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of *k* then left-out nodes, in the end, should remain as it is. https://leetcode.com/problems/reverse-nodes-in-k-group/ 2. Given the head of a singly linked list, return the middle node of the linked list. If there are two middle nodes, return the second middle node. https://leetcode.com/problems/middle-of-the-linked-list/ 3. Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.	2	**CLO – 1, 3**

	https://leetcode.com/problems/remove-duplicates-from-sorted-list/		
5.	**Topic:** Greedy algorithms 1. 1. Given an integer array *nums* of *2n* integers, group these integers into *n* pairs (a_1, b_1), (a_2, b_2), ..., (a_n, b_n) such that the sum of min(a_i, b_i) for all *i* is maximized. Return the maximized sum. https://leetcode.com/problems/array-partition/ 2. Given a string ***S***, return true if ***S*** can be a palindrome after deleting at most one character from it. https://leetcode.com/problems/valid-palindrome-ii/ 3. There are n points on a road you are driving your taxi on. The n points on the road are labelled from 1 to n in the direction you are going, and you want to drive from point 1 to point n to make money by picking up passengers. You cannot change the direction of the taxi. The passengers are represented by a 0-indexed 2D integer array rides, where rides[i] = [starti, endi, tipi] denotes the ith passenger requesting a ride from point starti to point endi who is willing to give a tipi dollar tip. For each passenger i you pick up, you earn endi - starti + tipi dollars. You may only drive at most one passenger at a time. Given n and rides, return the maximum number of dollars you can earn by picking up the passengers optimally. https://leetcode.com/problems/maximum-earnings-from-taxi/ 4. You are assigned to put some amount of boxes onto one truck. You are given a 2D array boxTypes, where boxTypes[i] = [numberOfBoxesi, numberOfUnitsPerBoxi]: numberOfBoxesi is the number of boxes of type i. numberOfUnitsPerBoxi is the number of units in each box of the type i. You are also given an integer truckSize, which is the maximum number of boxes that can be put on the truck. You can choose any boxes to put on the truck as long as the number of boxes does not exceed truckSize. Return the maximum total number of units that can be put on the truck. https://leetcode.com/problems/maximum-units-on-a-truck/description/	4	**CLO - 1, 3**
6.	**Topic:** Greedy algorithms Contd …	4	**CLO - 1, 3**
	Write a program to find the minimum spanning tree by using the greedy approach 1. Implement Kruskal's algorithm 2. Implement Prim's algorithm		
7.	**Topic:** Divide and Conquer 1. Given an array nums of size n, return the majority element. The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array. https://leetcode.com/problems/majority-element/	4	**CLO – 1, 3**

	2. Reverse bits of a given 32 bits unsigned integer. **Note:** Note that in some languages, such as Java, there is no unsigned integer type. In this case, both input and output will be given as a signed integer type. They should not affect your implementation, as the integer's internal binary representation is the same, whether it is signed or unsigned. In Java, the compiler represents the signed integers using 2's complement notation. Therefore, in Example 2 above, the input represents the signed integer - 3 and the output represents the signed integer -1073741825. https://leetcode.com/problems/reverse-bits/ 3. Your task is to calculate a^b mod 1337 where a is a positive integer and b is an extremely large positive integer given in the form of an array. https://leetcode.com/problems/super-pow/ 4. Given the head of a singly linked list where elements are sorted in ascending order, convert it to a height-balanced binary search tree. https://leetcode.com/problems/convert-sorted-list-to-binary-search-tree/ 5. Given an integer array nums, return the number of reverse pairs in the array. A reverse pair is a pair (i, j) where: 0 <= i < j < nums.length and nums[i] > 2 * nums[j]. https://leetcode.com/problems/reverse-pairs/		
8.	**Topic:** Divide and conquer contd... **1.** Write a program to implement the Quick Sort algorithm by randomly selecting any element of an array as the pivot element. Display the output after each call to the "PARTITION" function finishes. **2.** Write a program to implement merge sort using the divide-and-conquer approach.	**2**	**CLO – 1, 3**
9.	**Topic:** Dynamic Programming 1. You are given an array prices where prices[i] is the price of a given stock on the ith day. You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0. https://leetcode.com/problems/best-time-to-buy-and-sell-stock/ 2. Given an integer array nums, return the length of the longest strictly increasing subsequence. https://leetcode.com/problems/longest-increasing-subsequence/ 3. Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.	**4**	**CLO – 1, 3**

	You have the following three operations permitted on a word: Insert a character Delete a character Replace a character https://leetcode.com/problems/edit-distance/ 4. Given a sequence of matrices, find the most efficient way to multiply these matrices together. The efficient way is the one that involves the least number of multiplications. The dimensions of the matrices are given in an array arr[] of size N (such that N = number of matrices + 1) where the ith matrix has the dimensions (arr[i-1] x arr[i]). https://www.geeksforgeeks.org/dsa/matrix-chain-multiplication-dp-8/ 5. You are given an integer array coins representing coins of different denominations and an integer amount representing a total amount of money. Return the number of combinations that make up that amount. If that amount of money cannot be made up by any combination of the coins, return 0. https://leetcode.com/problems/coin-change-ii		
10.	**Topic:** Graph and Backtracking Algorithm 1. There is an undirected graph with n nodes, numbered from 0 to n - 1. You are given a 0-indexed integer array scores of length n where scores[i] denotes the score of node i. You are also given a 2D integer array edges where edges[i] = [ai, bi] denotes that there exists an undirected edge connecting nodes ai and bi. A node sequence is valid if it meets the following conditions: There is an edge connecting every pair of adjacent nodes in the sequence. No node appears more than once in the sequence. The score of a node sequence is defined as the sum of the scores of the nodes in the sequence. Return the maximum score of a valid node sequence with a length of 4. If no such sequence exists, return -1. https://leetcode.com/problems/maximum-score-of-a-node-sequence 2. Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses. https://leetcode.com/problems/generate-parentheses/	**2**	**CLO – 1, 3**
	Total	**30 hours**	

References:

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein , Introduction to Algorithm, PHI
2. Gills Brassard, Paul Bratley, Fundamental of Algorithms, PHI.
3. Dave and Dave, Design and Analysis of Algorithms, Pearson

4. Simen Harris, James Ross, Beginning Algorithms, Wiley India.
5. E.Horowitz and S. Sahni, Fundamentals of Computer Algorithms, Galgotia

Component wise Continuous Evaluation & Semester End Examination weightage:

Lesson Plan

Lecture No.	Sr. No.	Topics	Mapped CLOs	Applications
[02] 1 1	1	**ELEMENTARY ALGORITHMIC:** Introduction to Notation for programs. Efficiency of Algorithms. Average and worst-case analysis Elementary operation.	CLO 1,4	(i) Insertion Sort (ii) Selection Sort
[05] 1 2 2	2	**ASYMPTOTIC NOTATION:** A notation for "the order of" Other Asymptotic notations Conditional Asymptotic notation	CLO 1,4	(i) Bubble Sort (ii) Insertion Sort (iii) Selection Sort
[05] 2 2 1	3	**ANALYSIS OF ALGORITHMS:** Clamping operation, Analyzing control structures o Sequencing o "For" loops • Recursive calls • "While" and "repeat" loops • Using a barometer • Amortized analysis***[Self Study]***	CLO 1,4	(i) Binary Search (Iterative) (ii) Binary Search (Recursive) (iii) Euclid's algorithm (iv) Fibonacci Series
[08] 2 2 2 2	4	**GREEDY ALGORITHMS:** • Minimum Spanning Trees o Kruskal's Algorithm o Prim's Algorithm • Graphs: Shortest Paths • The Knapsack Problem • Scheduling o Minimizing time in the system o Scheduling with deadlines	CLO 2, 3	(i)Travelling Salesman Problem
[08] 1 1 2 2 2	5	**DIVIDE-AND-CONQUER:** • Multiplying large integers • Searching using Divide-and-Conquer • Sorting using Divide-and-Conquer • Finding the median • Matrix multiplication • Exponentiation ***[Self Study]***	CLO 2, 3	(i) Binary Search (ii)Quick Sort (iii) Merge Sort

[08] 1 1 2 2 1 1	**6**	**DYNAMIC PROGRAMMING:** Introduction Principle of optimality Knapsack Problem Shortest Path Approaches using Recursion Memory Functions	CLO 2, CLO 3	(i) Making Change (ii) Chained Matrix Multiplication
[07] **1** **2** **2** **2**	**7**	**BACKTRACKING & BRANCH-AND-BOUND:** Basic idea, 8-queens problem Graph coloring Hamiltonian cycles Knapsack problem	CLO 2, 3	Games Applications
[02] **1** **1**	**8**	**INTRODUCTION TO NP-COMPLETENESS**: Class P and NP, Polynomial reduction, NP- completeness problem, NP-hard problems	CLO 3,4	Research-based problems
45 hrs.		**Total**	**45 hrs**	

Tutorial Plan NA

Course Assessment Schemes :

(Course with Laboratory & Tutorial components)

Assessment scheme	**CE**		**LPW**		**SEE**
Component weightage	0.25		0.25		0.5
	Sessional Exam 60%	Innovative assignment 40%	Continuous Evaluation 60%	Viva Voce 40%	

Teaching-learning methodology: (Mention the proposed)

- Lectures: Use of Black board, PPT, Discussion, Case Studies etc.
- Laboratory: Implementation of various applications

Active learning techniques (Mention the proposed)

- Muddiest Points : For every topic the muddiest point will be identified and related to that the discussion is done. And Extra attention is given.
- Flipped classroom: Some exercises will be given and then discussions and interactively students illustrate the concept.

Types of Special/Innovative Assignments, Term Papers, mini Projects etc.

- 1. Objective of Assignment : Question answer
- 2. Assignment : Seminar topics are will be allocated and students are supposed to do through study on that topic and give the presentation.

Course Material:

- Course Policy
- PPTs, Notes, other Material
- Assignments, Tutorials, Lab Manuals
- Question bank
- Web-links, Blogs, Video Lectures, Journals
- Animations /Simulations, Softwares
- Advanced topics
- Industries/Organizations

Course Outcome Attainment:

- Use of formal evaluation components of continuous evaluation, tutorials, laboratory work, semester end examination
- Informal feedback during course conduction